Data & AI

11. Inside ResNet18

Dasaem Jeong · Art & Technology, Sogang University

Last Time

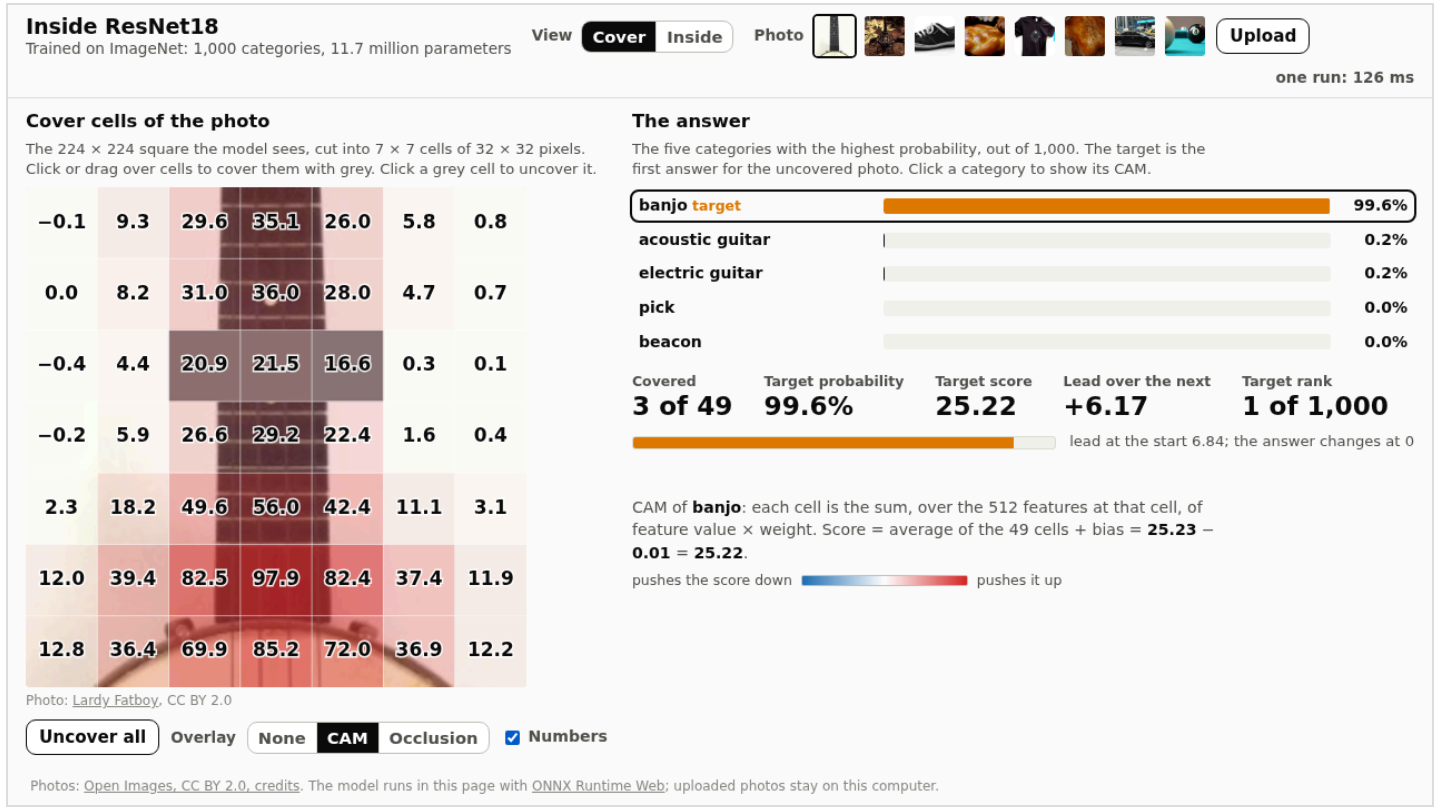
- A small CNN: convolution, max pooling, then 784×10
- ResNet18: 20 convolution layers, 11,689,512 parameters

Today

- Inside ResNet18: what changes its answer, and why
- Gradient ascent on the pixels: feature visualization, DeepDream, style transfer
- Adversarial examples

Inside ResNet18

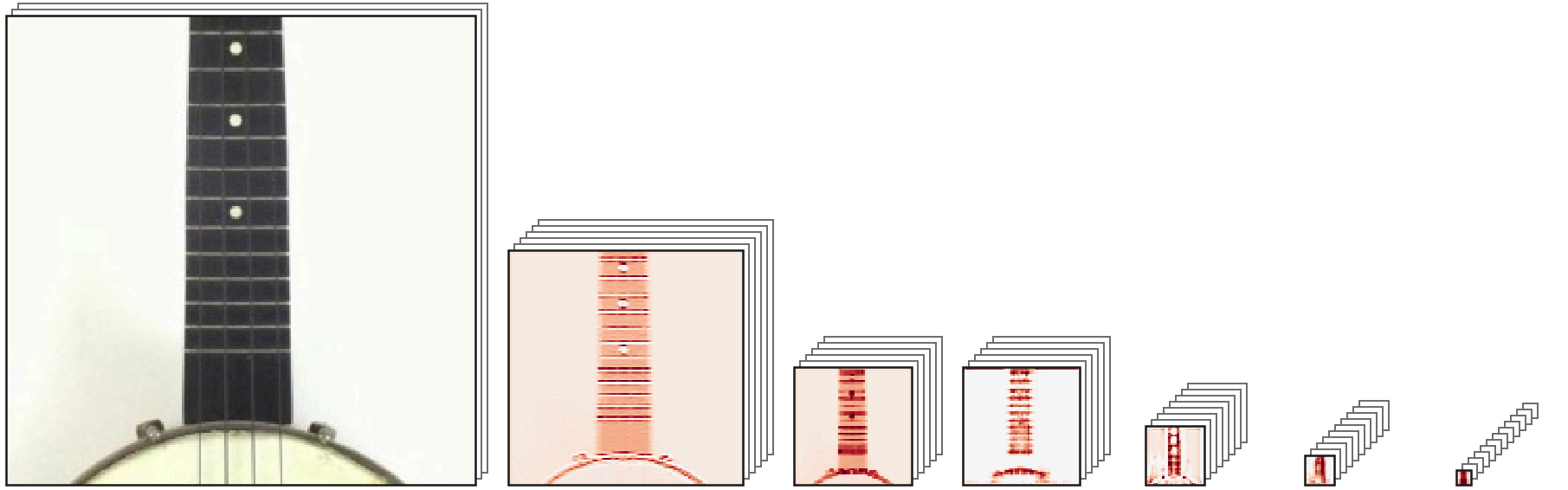
jdasam.github.io/mas1004/demos/lecture11-resnet18-cam



The Structure of ResNet18

ResNet18 Layer by Layer

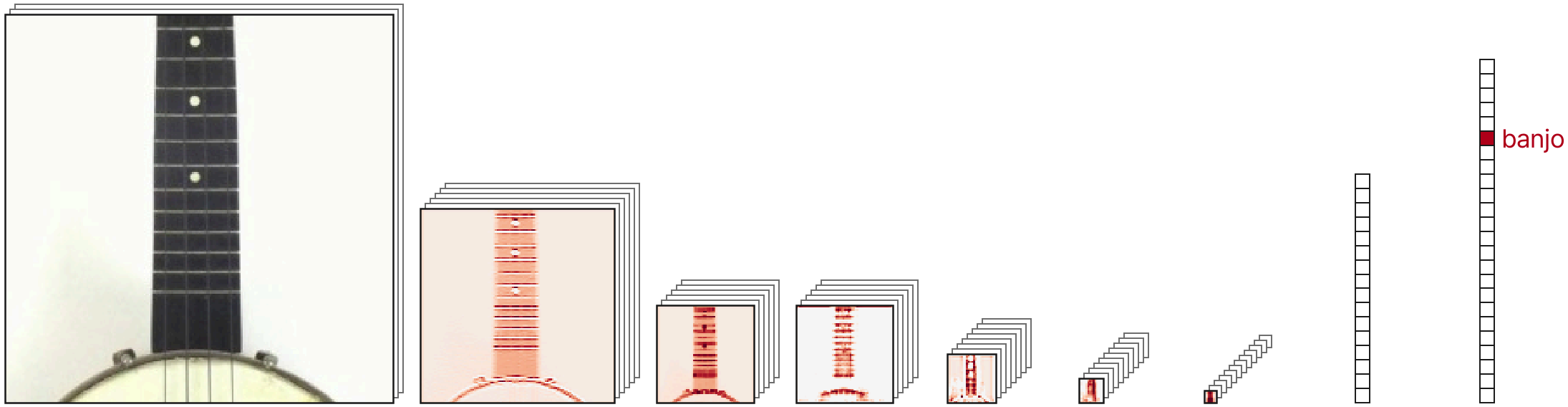
Output shape after each stage, for one 224 × 224 photo



input	conv1	max pool	layer1	layer2	layer3	layer4
224 × 224	112 × 112	56 × 56	56 × 56	28 × 28	14 × 14	7 × 7
3 colours	64 maps	64 maps	64 maps	128 maps	256 maps	512 maps

ResNet18 Layer by Layer

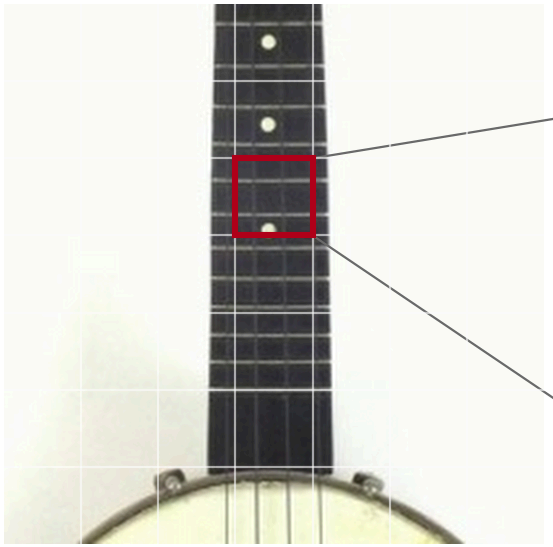
Output shape after each stage, for one 224 × 224 photo



Receptive Field

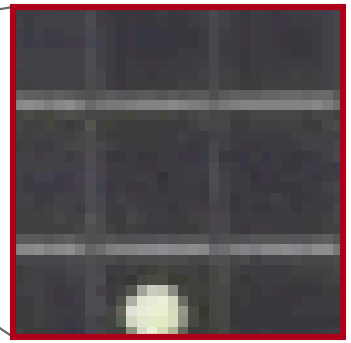
The pixels that can change one cell of layer4

7 × 7 cells



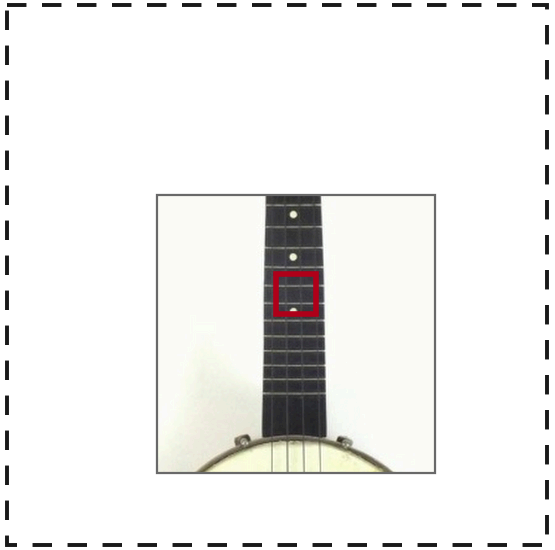
32 × 32 pixels each

row 3, column 4



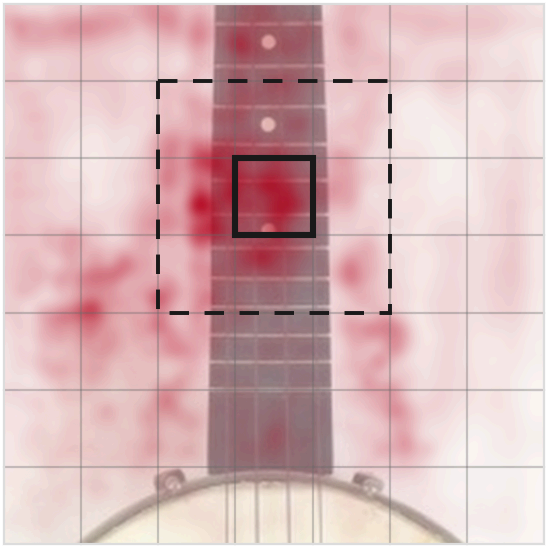
its 32 × 32 pixels

pixels that can change it



435 × 435 pixels,
more than the photo (224 × 224)

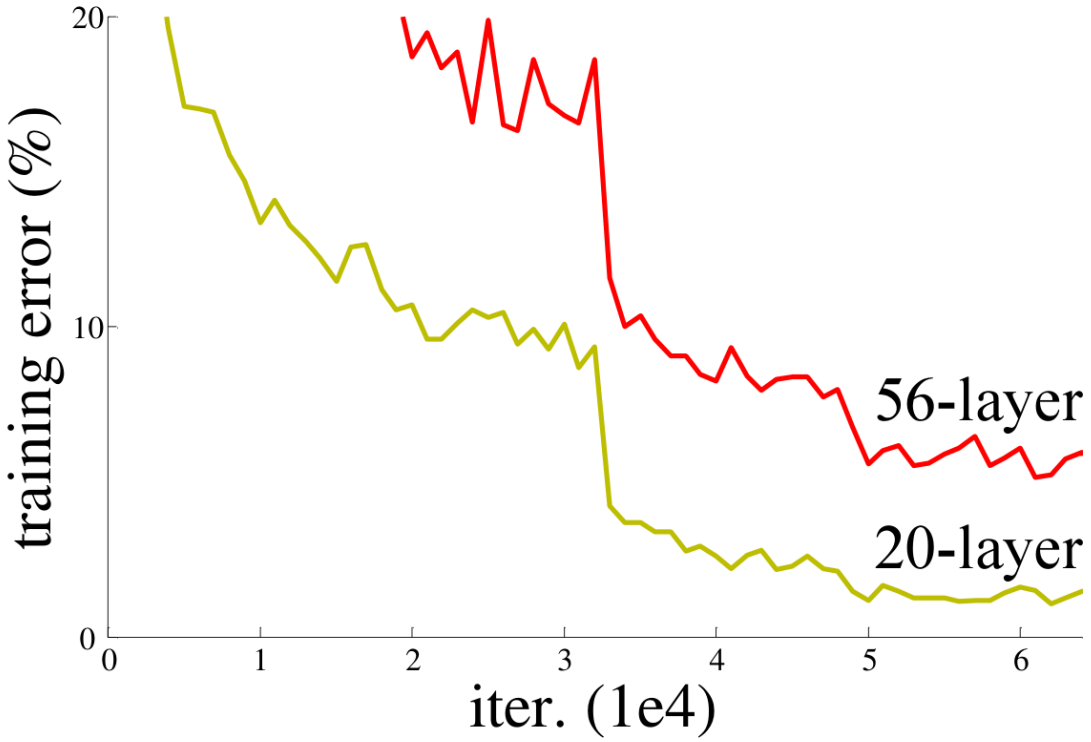
how much each pixel changes it

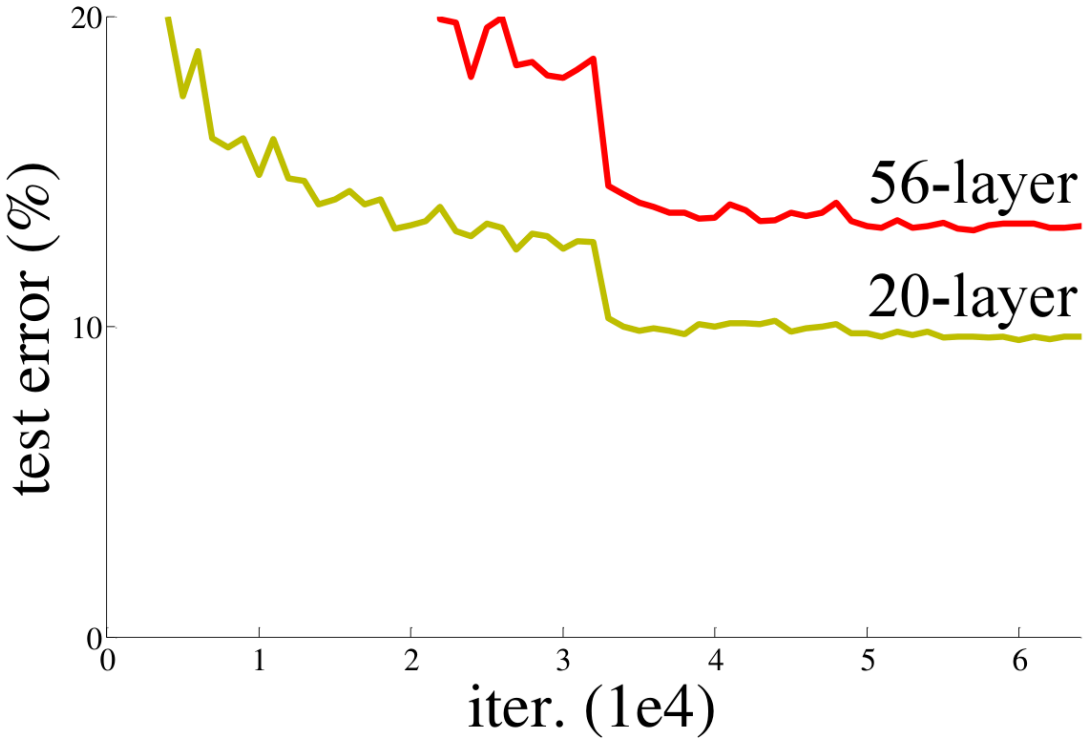


6% inside the cell,
33% inside the 3 × 3 cells

Training Deeper Networks

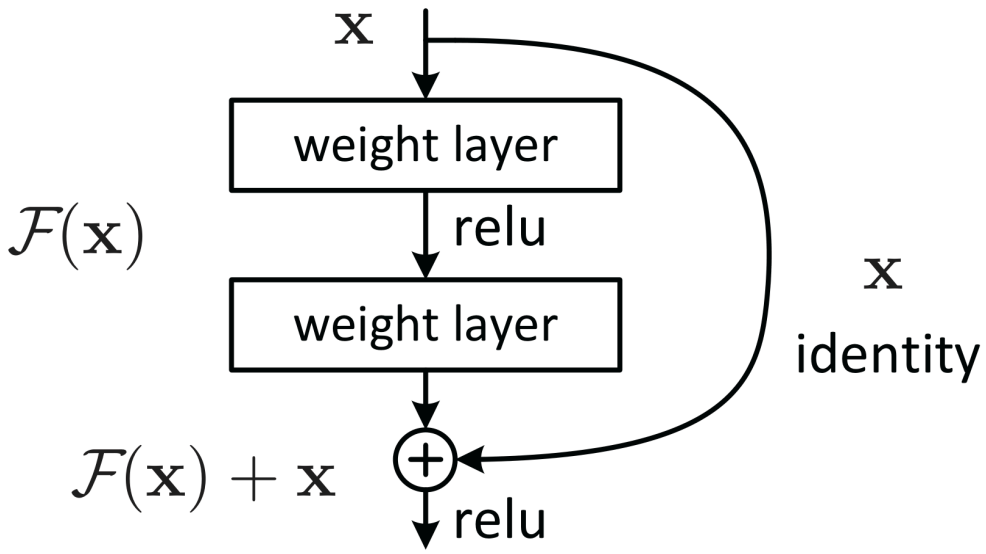
He et al. (2016): plain networks of 20 and 56 layers on CIFAR-10





Residual Block

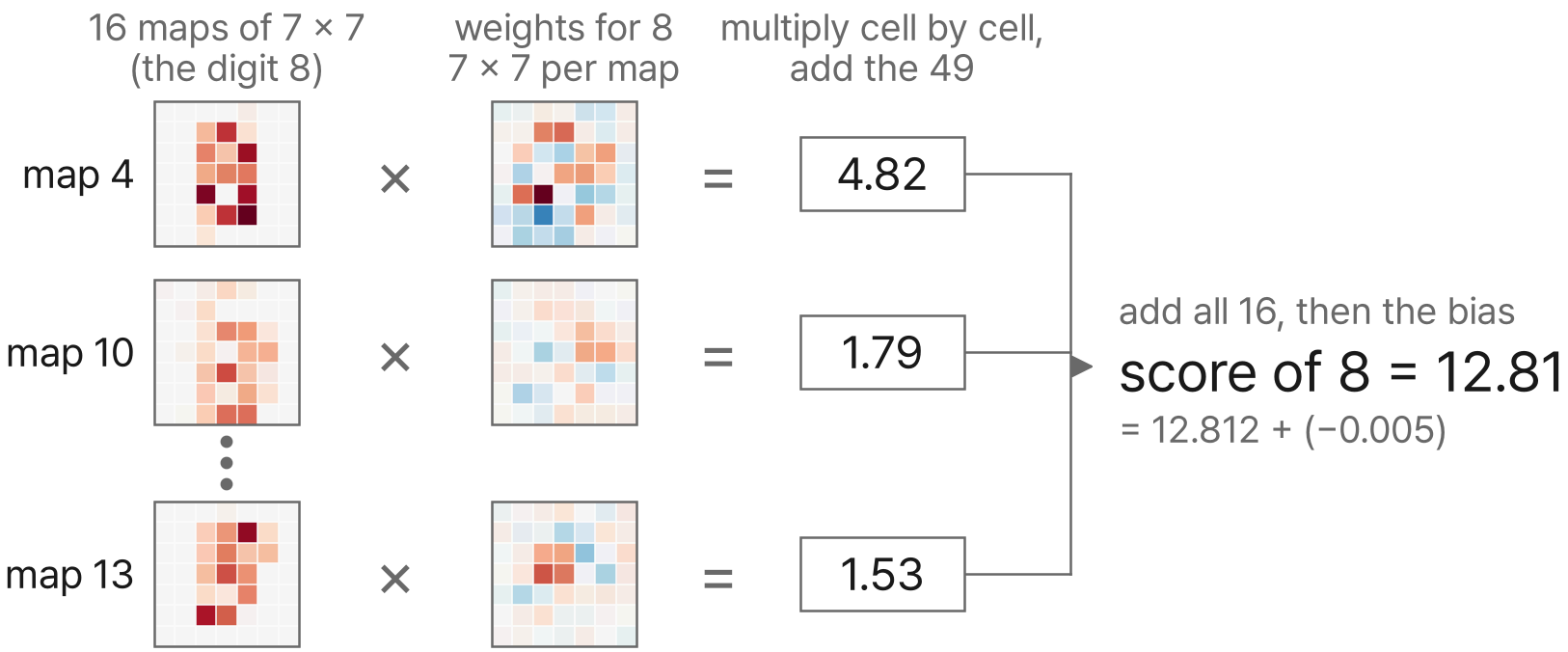
He, Zhang, Ren, Sun (2016), Deep Residual Learning for Image Recognition, CVPR



- ResNet18: 8 blocks of two 3 × 3 convolutions

The Last Layer of the Small CNN for MNIST

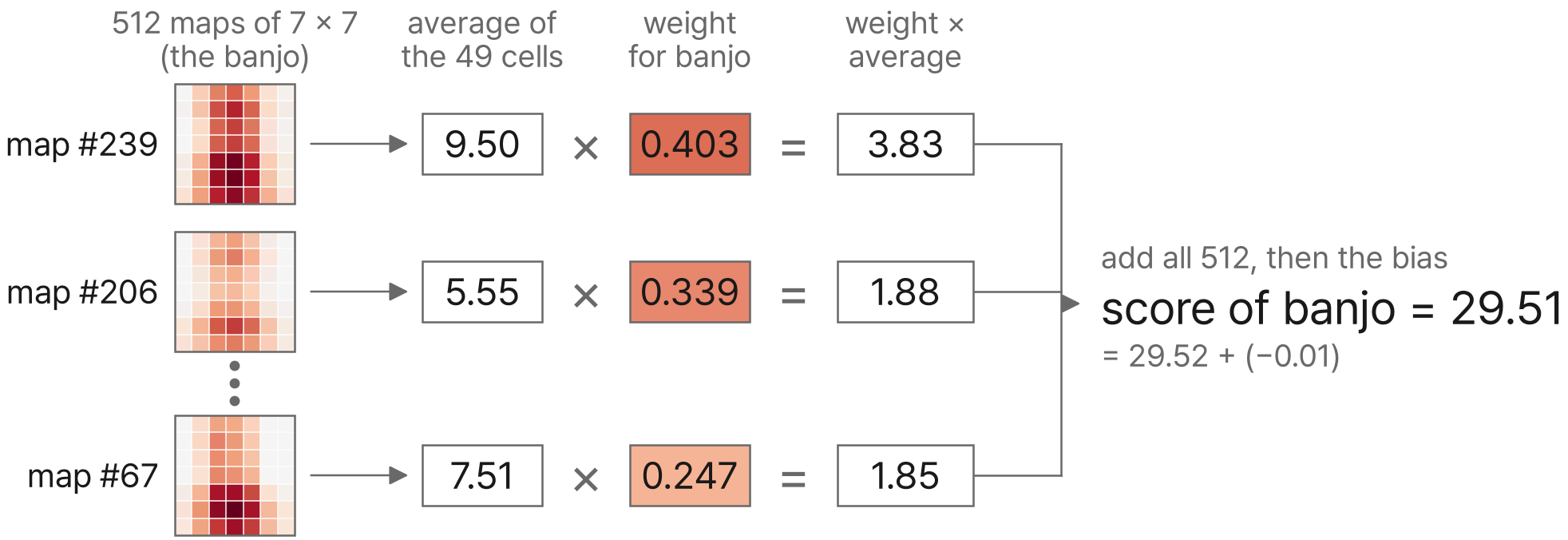
Lecture 9, handwritten digits: 16 maps of 7 × 7 flattened into 784 numbers, then 10 scores



Flattened: 16 × 7 × 7 = 784 numbers, each with its own weight
784 weights × 10 digits = 7,840 weights

Global Average Pooling

ResNet18: each 7 × 7 map averaged into one number, then 512 × 1,000



Averaged: 512 maps become 512 numbers, one weight each (the same for all 49 cells)
512 weights × 1,000 categories = 512,000 weights

Splitting the Score by Cell

$f_k(c)$: value of feature k at cell c · w_k: its weight for the category

$$\text{score} = \sum_{k=1}^{512} w_k \cdot \frac{1}{49} \sum_{c=1}^{49} f_k(c) \; + \; b$$

Splitting the Score by Cell

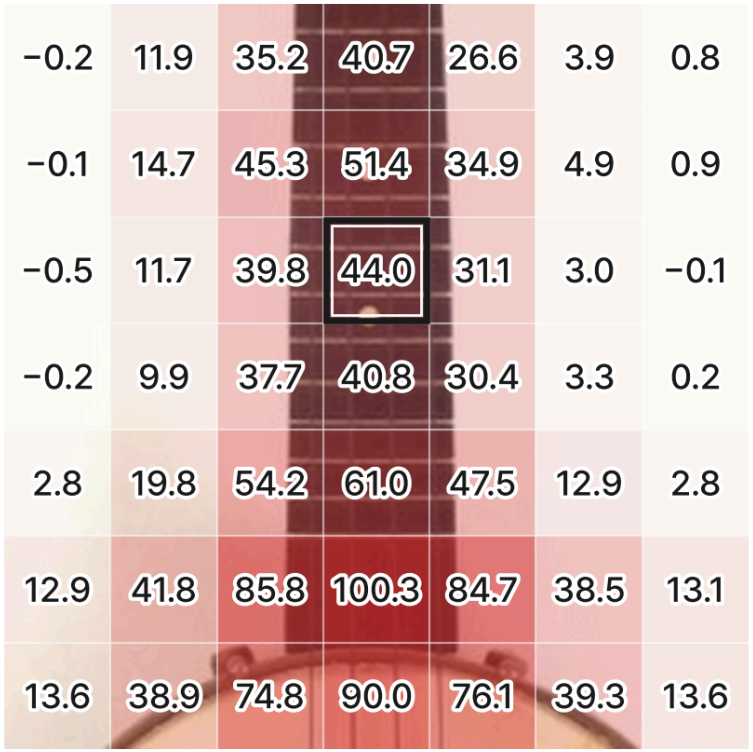
$f_k(c)$: value of feature k at cell c · w_k: its weight for the category

$$\text{score} = \sum_{k=1}^{512} w_k \cdot \frac{1}{49} \sum_{c=1}^{49} f_k(c) \; + \; b$$

$$= \frac{1}{49} \sum_{c=1}^{49} \underbrace{\sum_{k=1}^{512} w_k f_k(c)}_{\text{value of cell } c} \; + \; b$$

Class Activation Map

Zhou, Khosla, Lapedriza, Oliva, Torralba (2016), Learning Deep Features for Discriminative Localization, CVPR



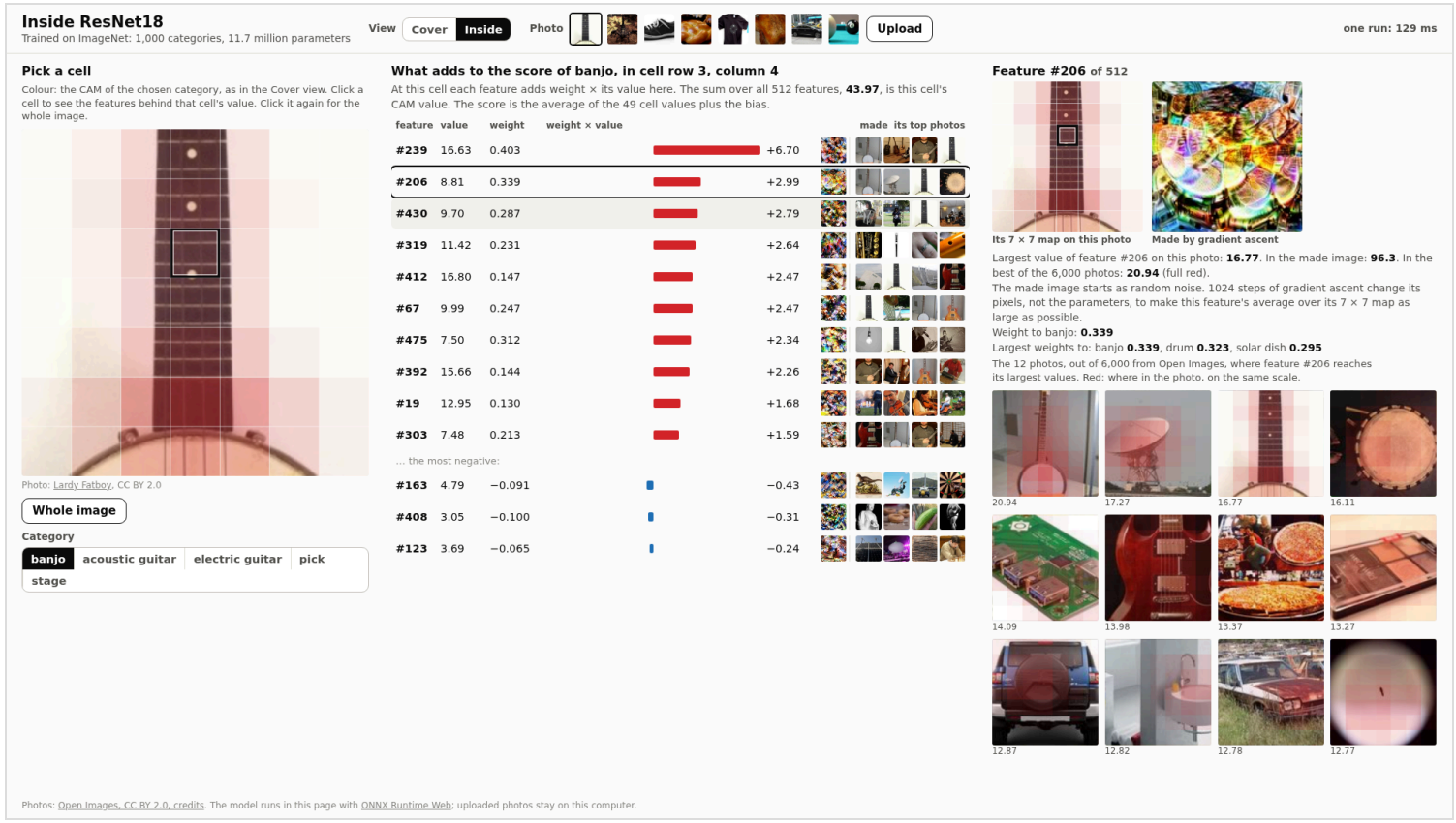
In each cell: weight × value, added over the 512 features

average of the 49 cells	29.52
bias	−0.01
score of banjo	29.51

pushes the score down — pushes it up

Splitting a Cell by Feature

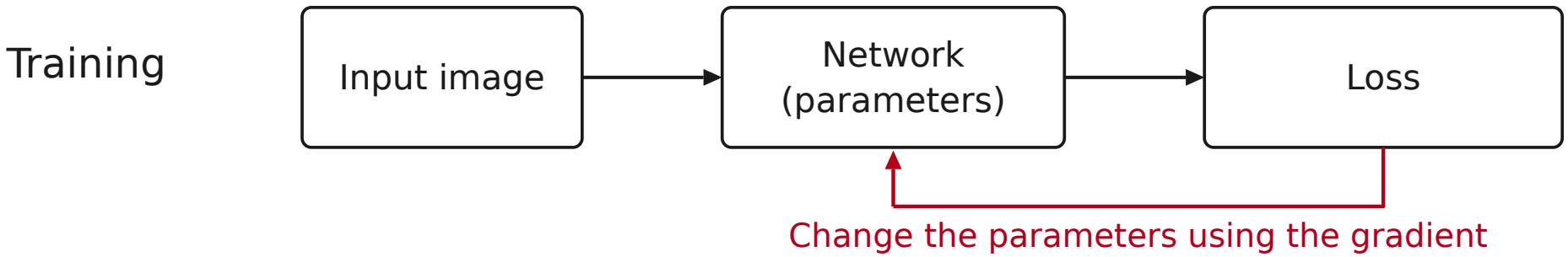
Inside view: banjo, cell at row 3, column 4



Gradient Ascent on the Pixels

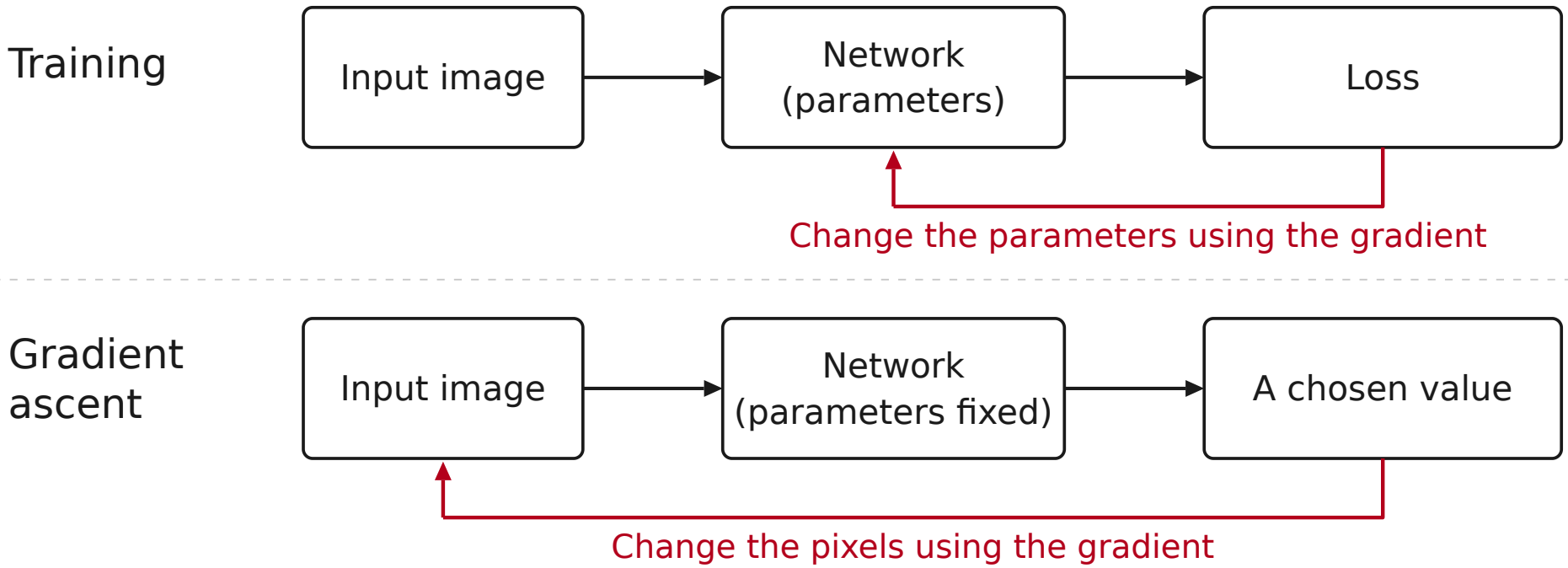
Gradients for the Pixels

Backprop gives a gradient for every input number too



Gradients for the Pixels

Backprop gives a gradient for every input number too

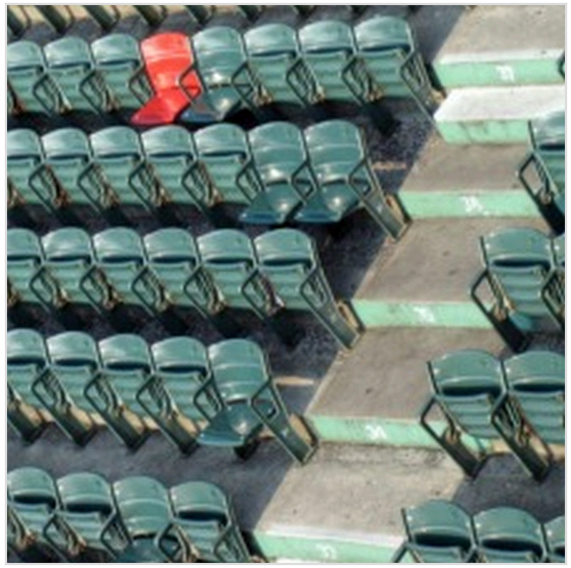


Feature Visualization

Olah, Mordvintsev, Schubert (2017), Feature Visualization, Distill

Best of 6,000 photos

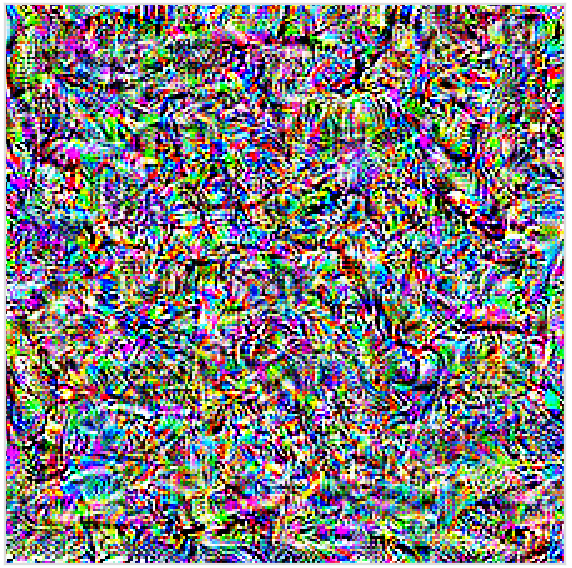
the photo where this feature is largest



feature #194: 10.4

Plain gradient ascent

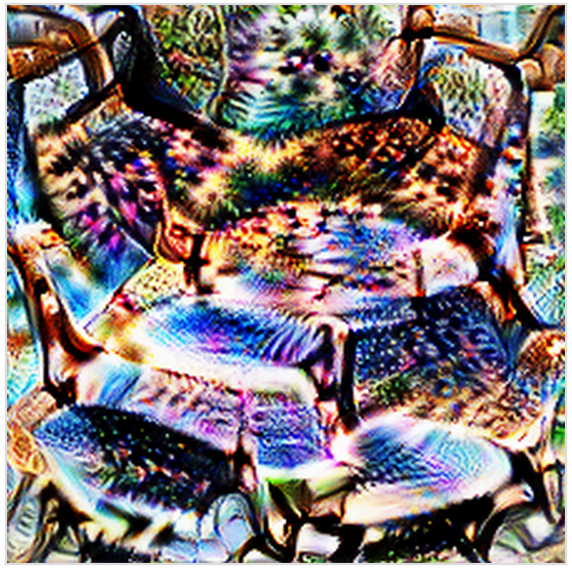
start: random pixels, 1,024 steps



feature #194: 195.9

Regularized gradient ascent

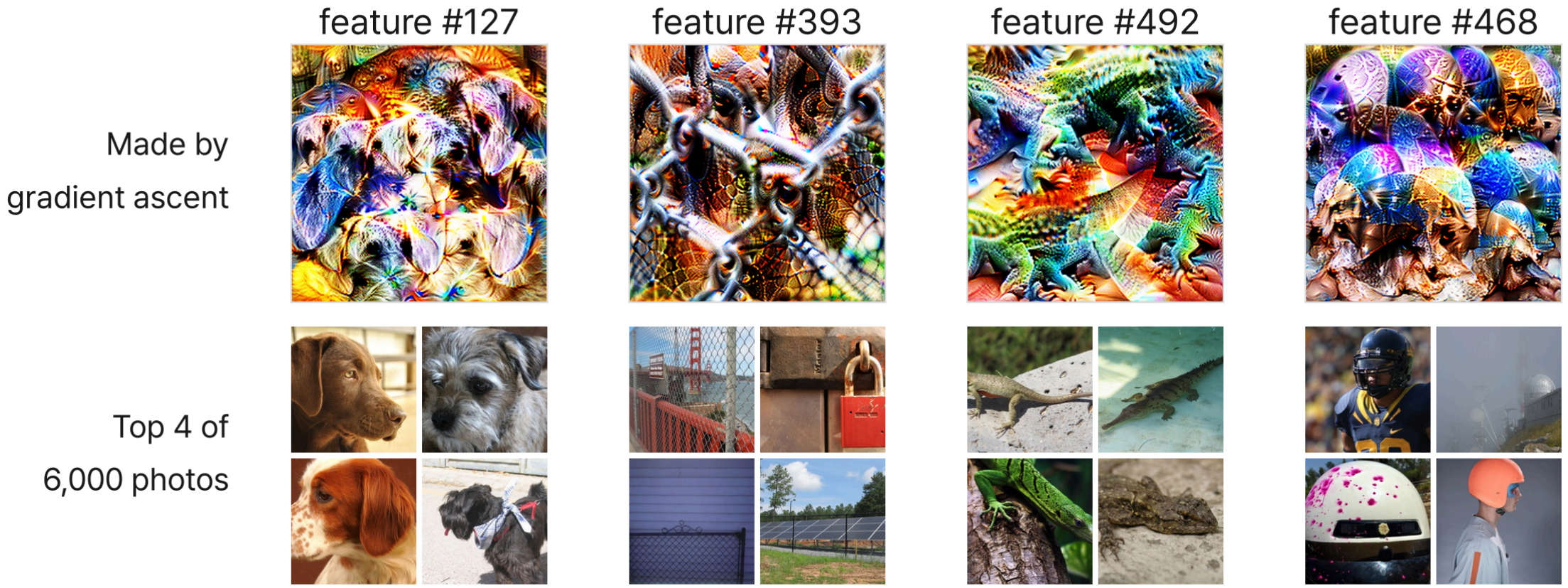
smooth pixels, random shift, scale, rotation



feature #194: 71.0

Features and Their Top Photos

Made by gradient ascent, and the 4 photos out of 6,000 where the feature is largest



Photos: Open Images V7, CC BY 2.0, credits

Class Visualization

Mordvintsev, Olah, Tyka (2015), Inceptionism: Going Deeper into Neural Networks, Google Research Blog

Class Visualization with ResNet18

Gradient ascent on the score of one category

dumbbell

score 142
probability 0.99

banjo

score 200
probability 1.00

pool table

score 142
probability 1.00

cab

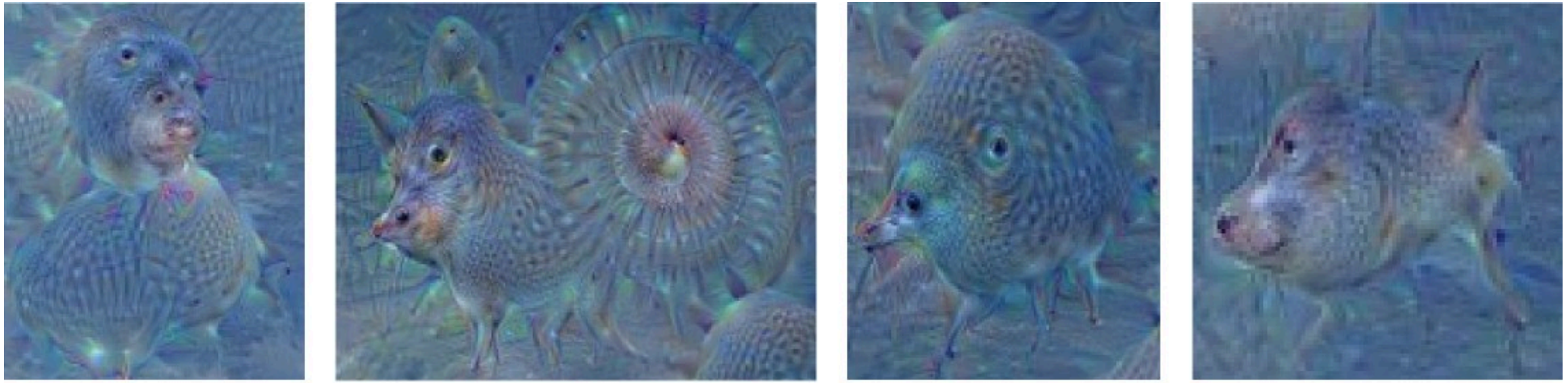
score 108
probability 1.00

zebra

score 114
probability 1.00

DeepDream

Mordvintsev, Olah, Tyka (2015), Inceptionism: Going Deeper into Neural Networks, Google Research Blog

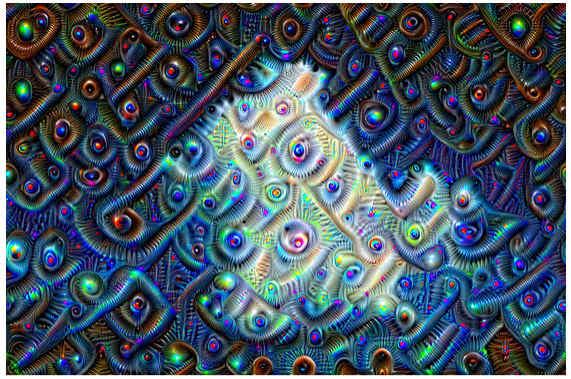


- Start from a photo, and make the values of a whole layer larger

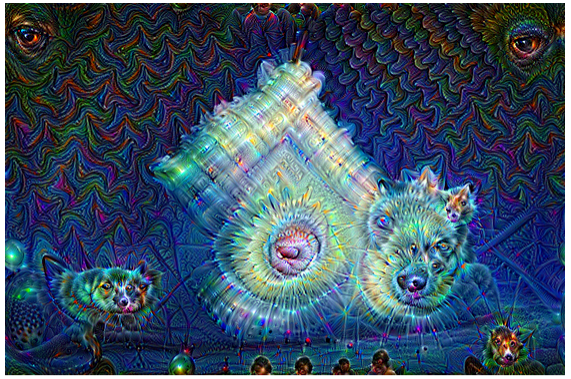
DeepDream by Layer

ResNet18, the same photo

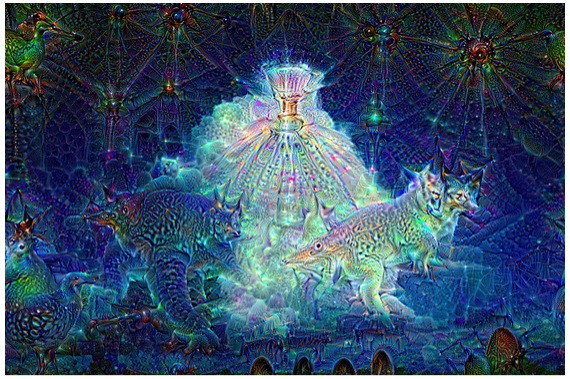
Input photo



layer2 (shallow)



layer3 (middle)

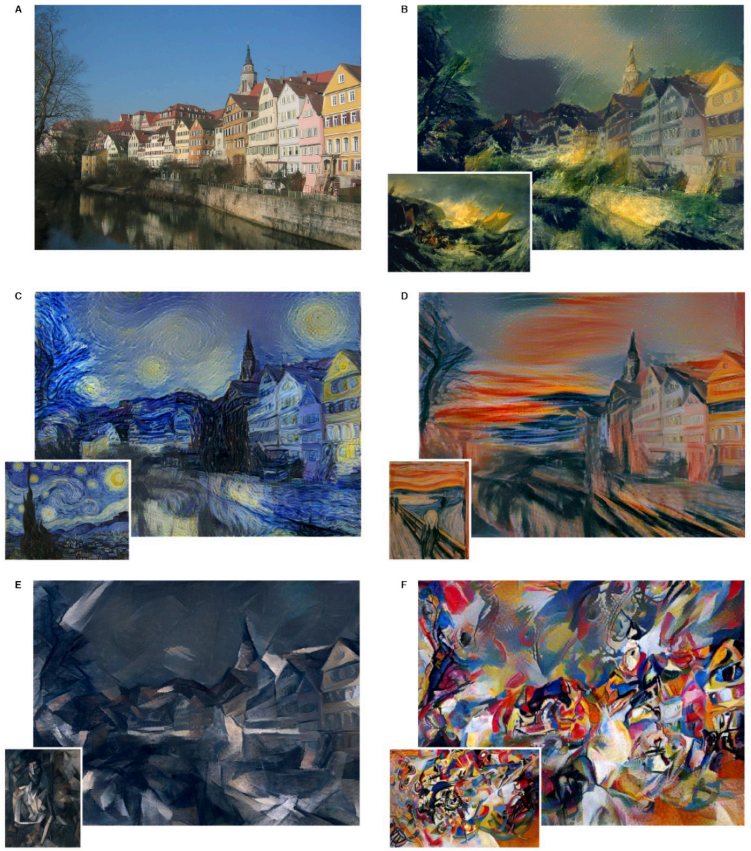


layer4, first block (deep)

Input: Ralph F. Kresge, Cumulus6 (NOAA, public domain)

Neural Style Transfer

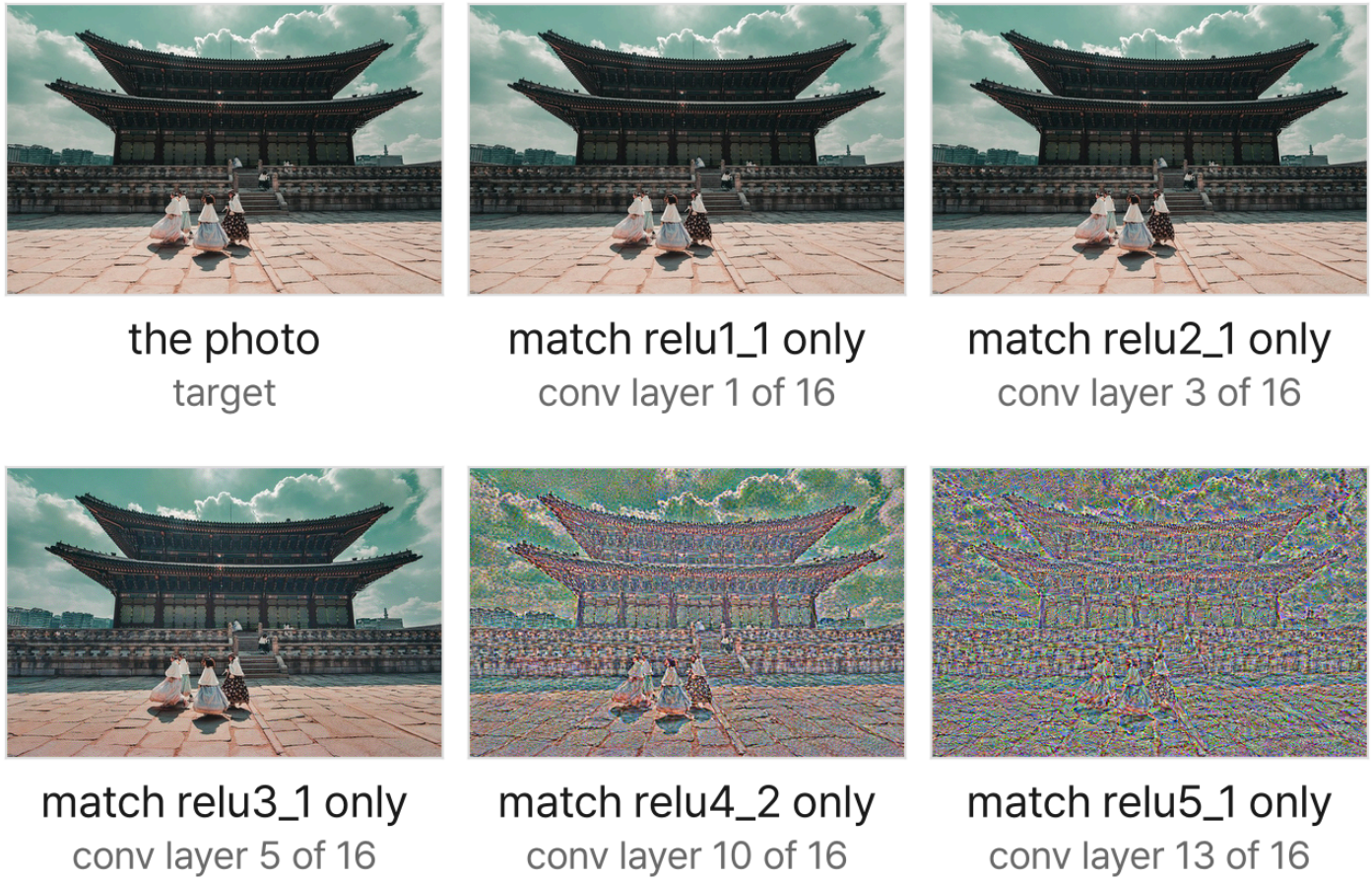
Gatys, Ecker, Bethge (2016), Image Style Transfer Using Convolutional Neural Networks, CVPR



- Content loss: the maps of a deep layer, cell by cell
- Style loss: which features appear together, position thrown away
- Change the pixels to lower the sum of the two losses

Content Reconstruction

VGG19: start from noise and match the maps of one layer to the photo



Gatys, Ecker, Bethge (2016), CVPR · Photo: chsyang, Gyeongbokgung Geunjeongjeon (Public Domain Mark)

The Style Table

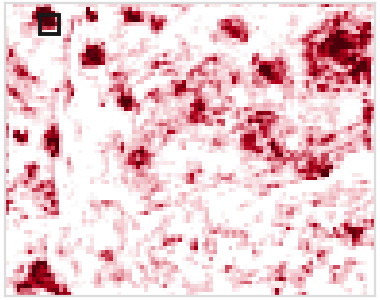
Gram matrix: two maps multiplied cell by cell, then averaged over all cells

The Starry Night

VGG19, layer relu4_1

feature #407

high on stars, moon



its map: 76 × 96 cells

boxed cell: 12.1

The Style Table

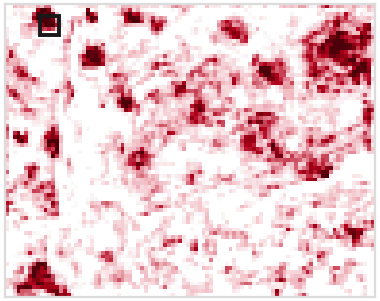
Gram matrix: two maps multiplied cell by cell, then averaged over all cells

The Starry Night

VGG19, layer relu4_1

feature #407

high on stars, moon

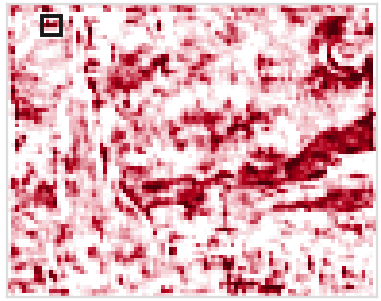


its map: 76 × 96 cells

boxed cell: 12.1

feature #15

high on sky swirls



its map: 76 × 96 cells

boxed cell: 4.6

The Style Table

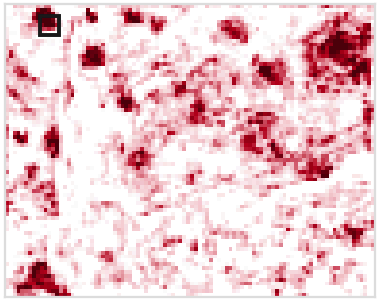
Gram matrix: two maps multiplied cell by cell, then averaged over all cells

The Starry Night

VGG19, layer relu4_1

feature #407

high on stars, moon



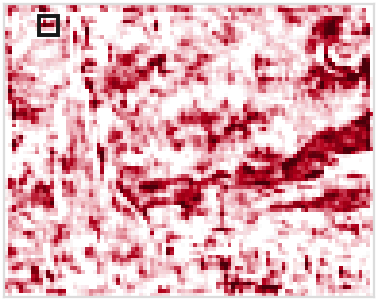
its map: 76 × 96 cells

boxed cell: 12.1

×

feature #15

high on sky swirls



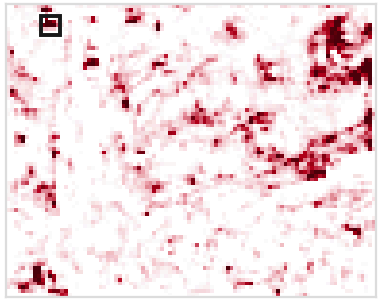
its map: 76 × 96 cells

boxed cell: 4.6

=

#407 × #15

cell by cell



high where both are high

12.1 × 4.6 = 55.3

The Style Table

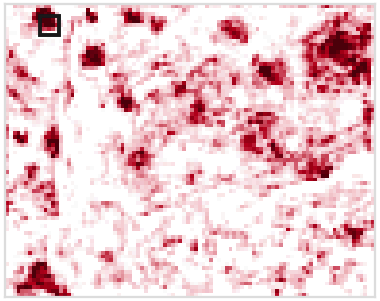
Gram matrix: two maps multiplied cell by cell, then averaged over all cells

The Starry Night

VGG19, layer relu4_1

feature #407

high on stars, moon



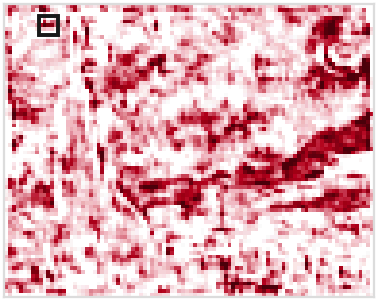
its map: 76 × 96 cells

boxed cell: 12.1

×

feature #15

high on sky swirls



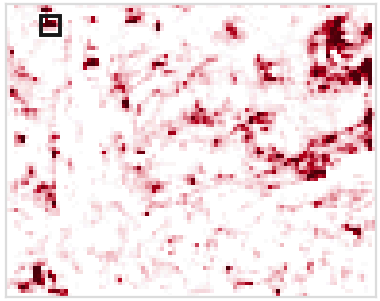
its map: 76 × 96 cells

boxed cell: 4.6

=

#407 × #15

cell by cell



high where both are high

12.1 × 4.6 = 55.3

average of all 7,296 cells

= 2.91

The Style Table

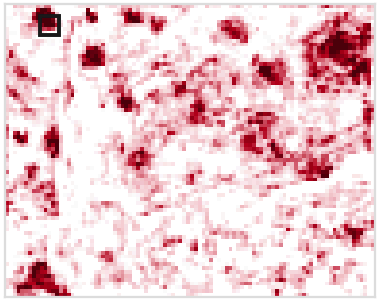
Gram matrix: two maps multiplied cell by cell, then averaged over all cells

The Starry Night

VGG19, layer relu4_1

feature #407

high on stars, moon



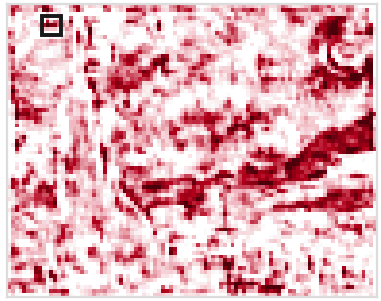
its map: 76 × 96 cells

boxed cell: 12.1

×

feature #15

high on sky swirls



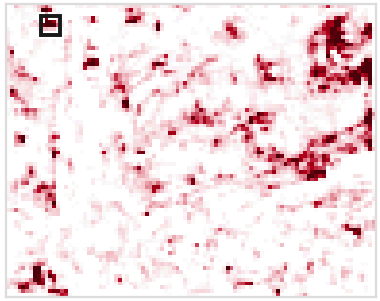
its map: 76 × 96 cells

boxed cell: 4.6

=

#407 × #15

cell by cell



high where both are high

12.1 × 4.6 = 55.3

average of all 7,296 cells

= 2.91

style table

(Gram matrix)

	#407	#15	#422
#407	4.83	2.91	0.24
#15	2.91	4.63	0.30
#422	0.24	0.30	0.66

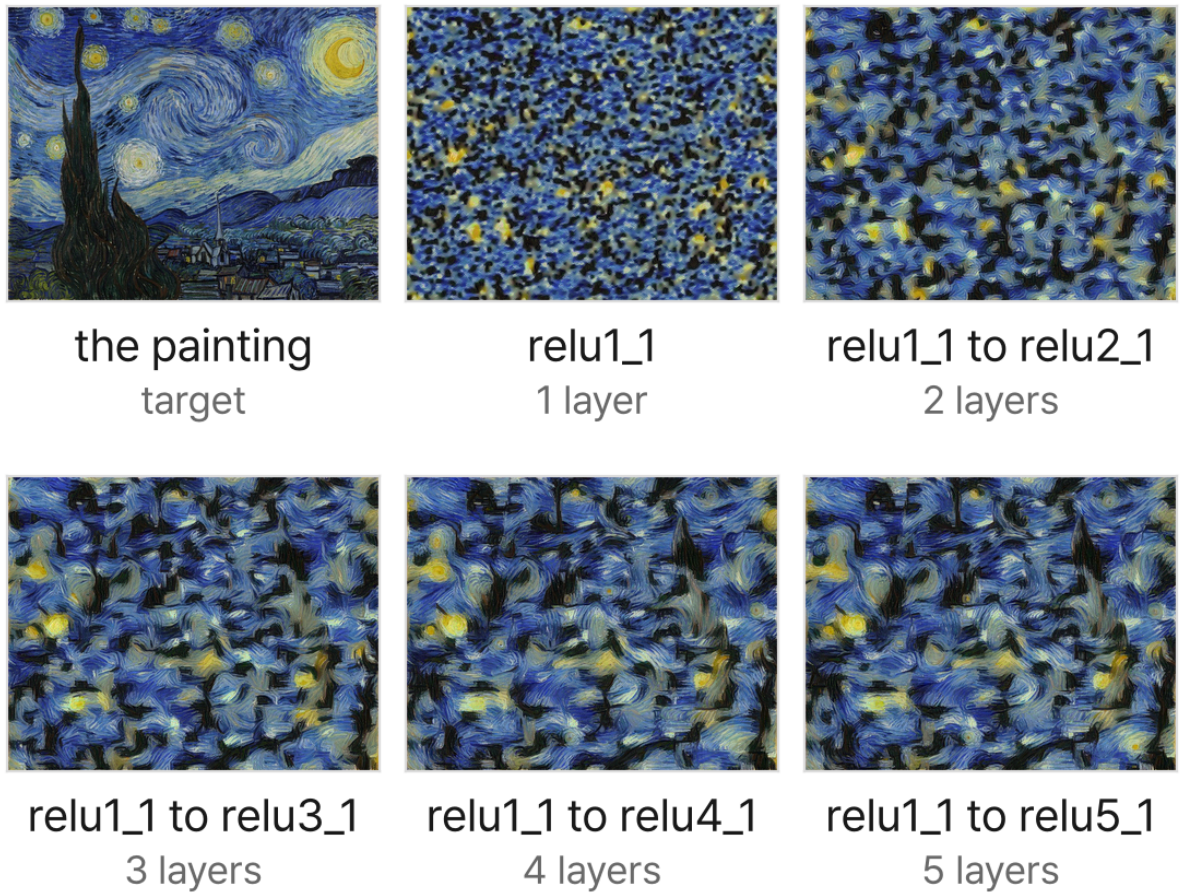
#422: edge of the cypress

full table: 512 × 512 numbers

Averaging over all cells throws away where things are. The table keeps only which features appear together, and how strongly.

Style Reconstruction

VGG19: start from noise and match only the style tables of the painting



Gatys, Ecker, Bethge (2016), CVPR · Painting: Vincent van Gogh, The Starry Night

A Shuffled Painting

The Starry Night cut into 4 × 4 pieces and shuffled

The Starry Night

cut into 4 × 4 pieces, shuffled

change from the original
(difference ÷ size of the original)

style tables, 5 layers
3% to 20%

content features (relu4_2)
131%

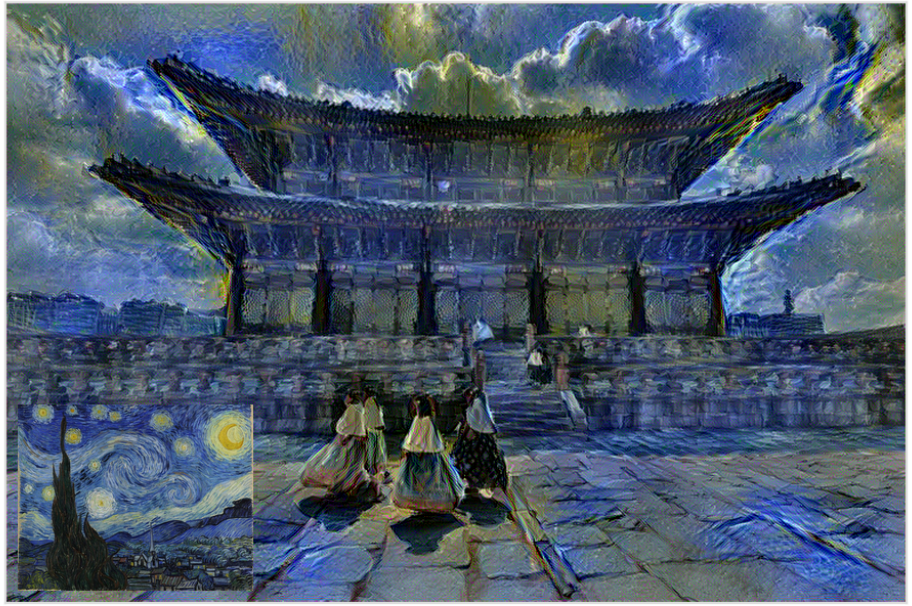
for scale, the palace photo
vs the painting: style tables 48% to 107%

Painting: Vincent van Gogh, The Starry Night

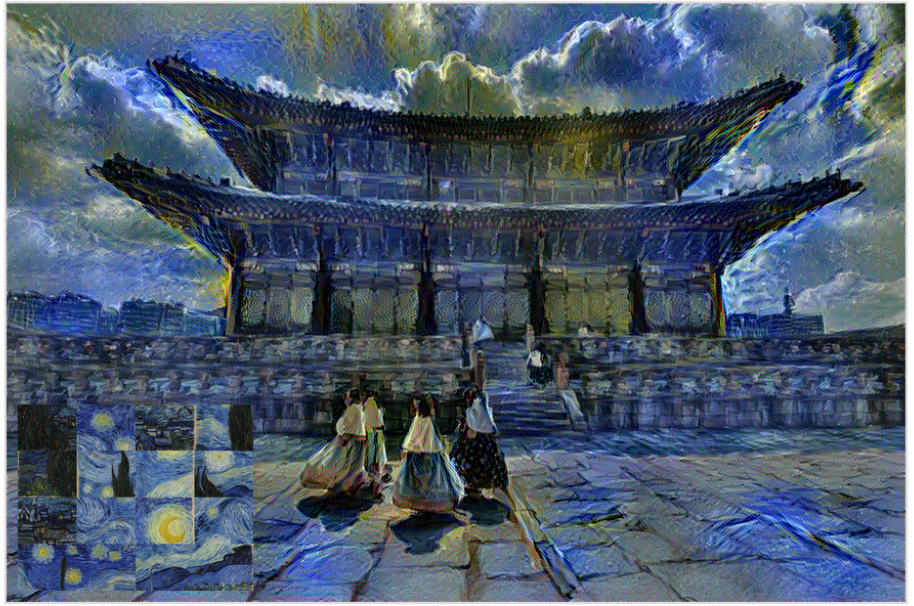
Style from the Shuffled Painting

The same photo, starting noise and settings, only the style image differs

style from the painting



style from the shuffled painting



Average pixel difference between the two results: 19 of 255

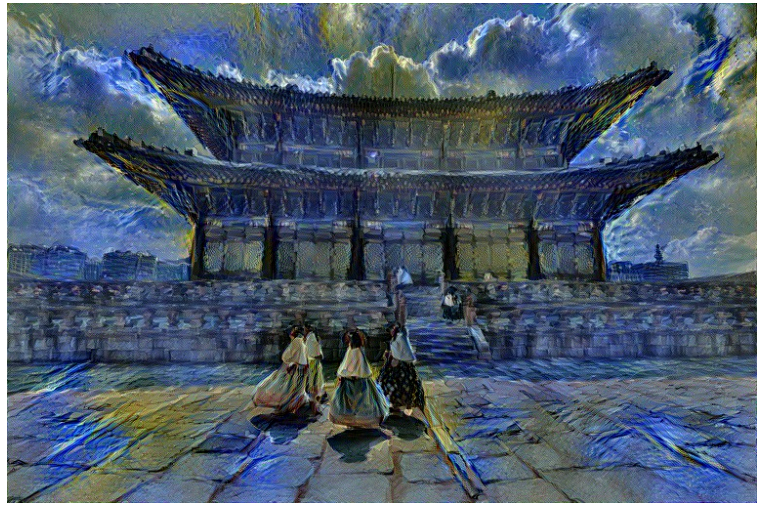
(same painting, started from different noise: 36 of 255)

Style Transfer Result

VGG19 · Gyeongbokgung photo and The Starry Night

Content

Style



Result after 400 steps from noise

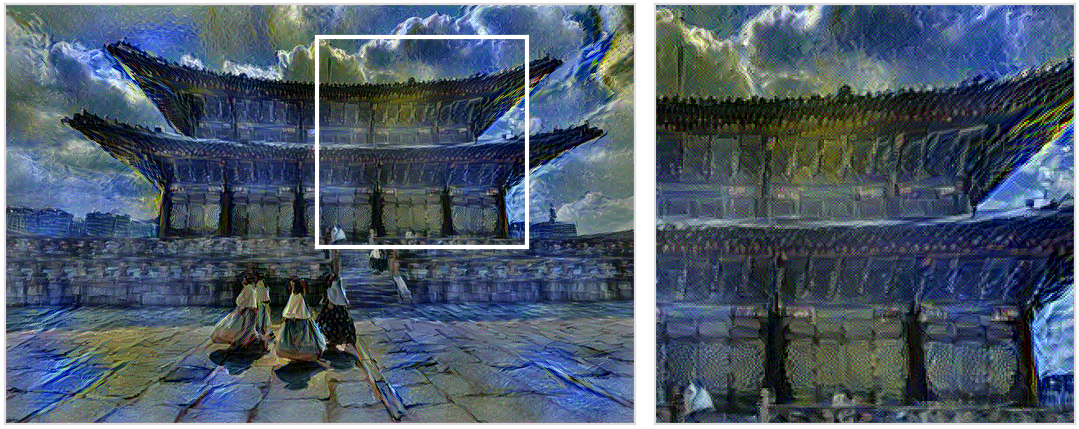
Content: chsyang, Gyeongbokgung Geunjeongjeon (Public Domain Mark) · Style: Vincent van Gogh, The Starry Night

Style Transfer with ResNet18

The same losses with the model from our demo

VGG19

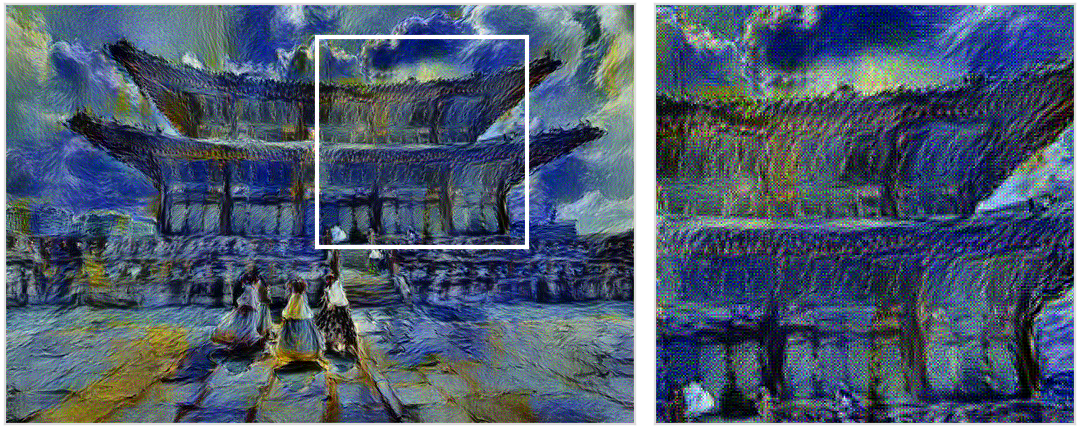
used by Gatys et al. (2015)



boxed part, enlarged

ResNet18

the model from our demo



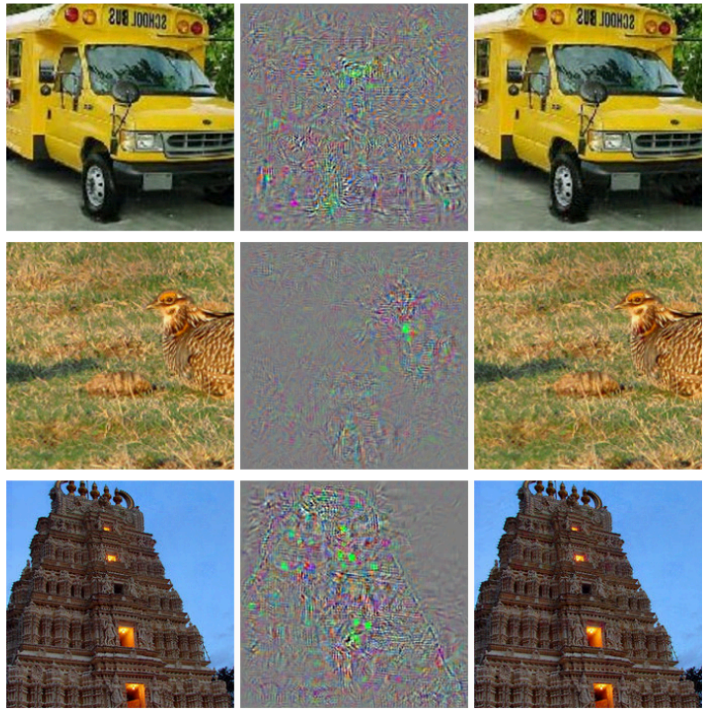
boxed part, enlarged

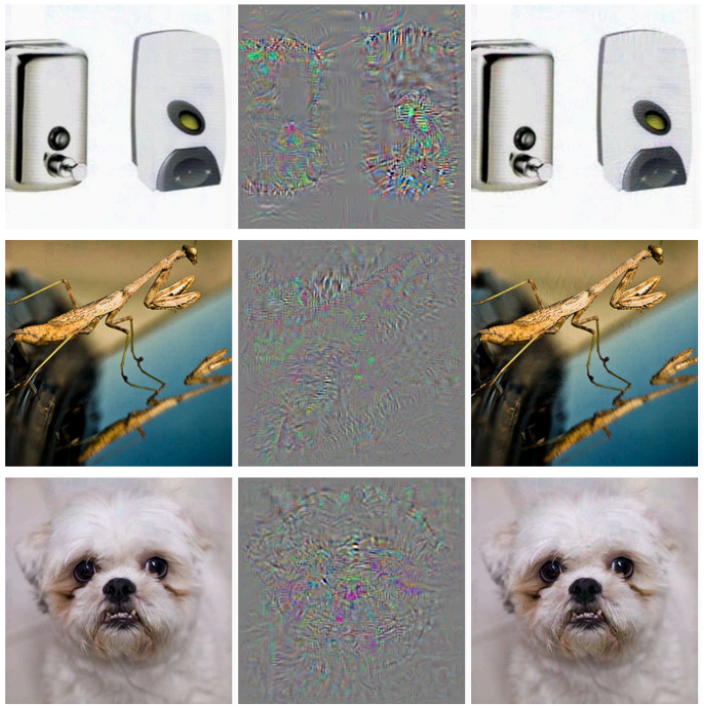
Wang, Li, Vasconcelos (2021), Rethinking and Improving the Robustness of Image Style Transfer, CVPR

Adversarial Examples

Adversarial Examples

Szegedy et al. (2014), Intriguing Properties of Neural Networks, ICLR





- The same gradient for the pixels, used to change the answer

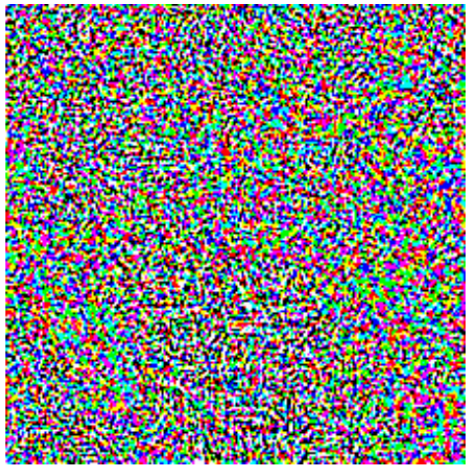
Fast Gradient Sign Method

Goodfellow, Shlens, Szegedy (2015), Explaining and Harnessing Adversarial Examples, ICLR

$\boldsymbol{x}$

"panda"
57.7% confidence

$+ .007 \times$



$\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$

"nematode"
8.2% confidence

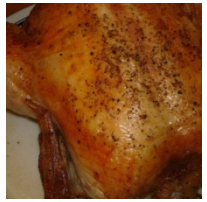
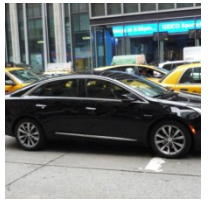
$=$

$\boldsymbol{x} + \epsilon\text{sign}(\nabla_{\boldsymbol{x}} J(\boldsymbol{\theta}, \boldsymbol{x}, y))$

"gibbon"
99.3 % confidence

FGSM on the Demo Photos

ResNet18, one step against the correct answer, each value moved by 1, 2 or 4 of 255

								
original	banjo 99.8%	scorpion 99.9%	running shoe 99.6%	French loaf 86.8%	jersey 95.0%	rotisserie 94.0%	cab 98.2%	pool table 98.6%
1 of 255	banjo 83.3%	badger 18.6%	sandal 80.3%	pretzel 83.4%	academic gown 81.1%	sea cucumber 28.9%	sports car 38.0%	ping-pong ball 96.9%
2 of 255	banjo 58.2%	pug 17.1%	sandal 90.3%	pretzel 76.5%	academic gown 77.2%	starfish 38.0%	sports car 48.5%	ping-pong ball 95.7%
4 of 255	banjo 46.9%	pug 53.5%	sandal 92.4%	pretzel 60.5%	academic gown 69.7%	starfish 60.3%	sports car 39.8%	ping-pong ball 79.9%

Photos: Open Images V7, CC BY 2.0, credits

An Adversarial Banjo

ResNet18, the banjo from the demo, each pixel changed by at most 1 level out of 255

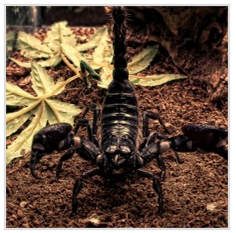
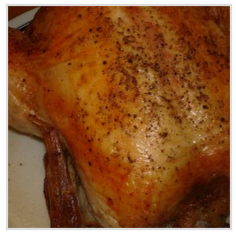
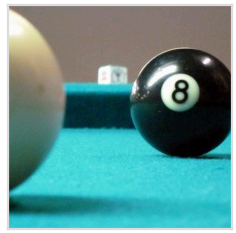
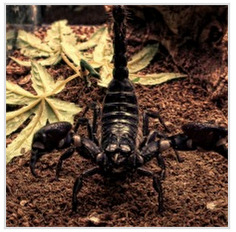
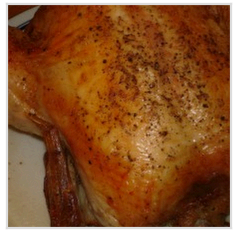
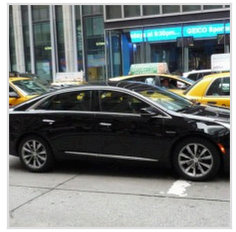
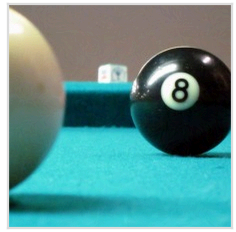
banjo 99.8%

Change × 60

toaster 100%

Adversarial Examples for the Demo Photos

ResNet18, each value changed by at most 1 of 255, toward a chosen category

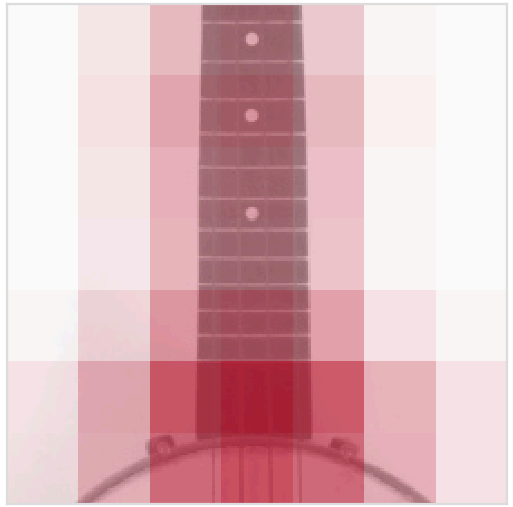
original								
	banjo 99.8%	scorpion 99.9%	running shoe 99.6%	French loaf 86.8%	jersey 95.0%	rotisserie 94.0%	cab 98.2%	pool table 98.6%
each value changed by at most 1 of 255								
	toaster >99.9%	teddy 99.9%	banana >99.9%	goldfish >99.9%	space shuttle >99.9%	ostrich >99.9%	giant panda 99.7%	volcano >99.9%

Photos: Open Images V7, CC BY 2.0, credits

CAM of the Adversarial Banjo

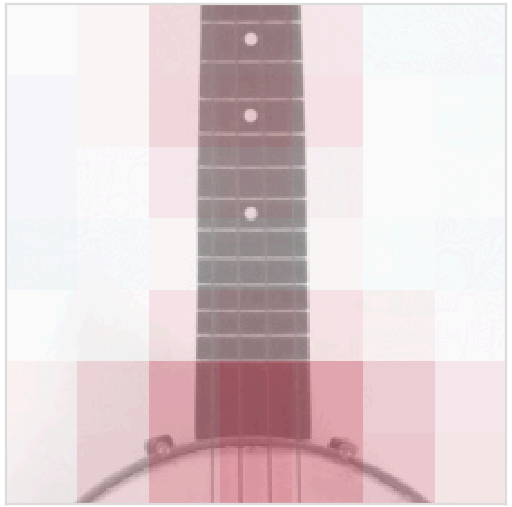
The same model, before and after the change

banjo, original photo



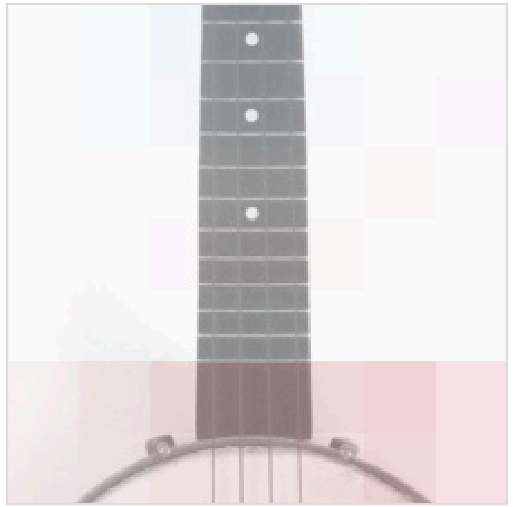
score 29.5

banjo, changed photo



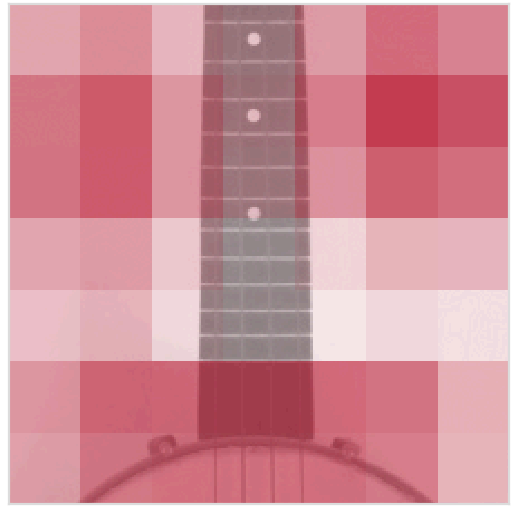
score 11.4

toaster, original photo



score 2.0

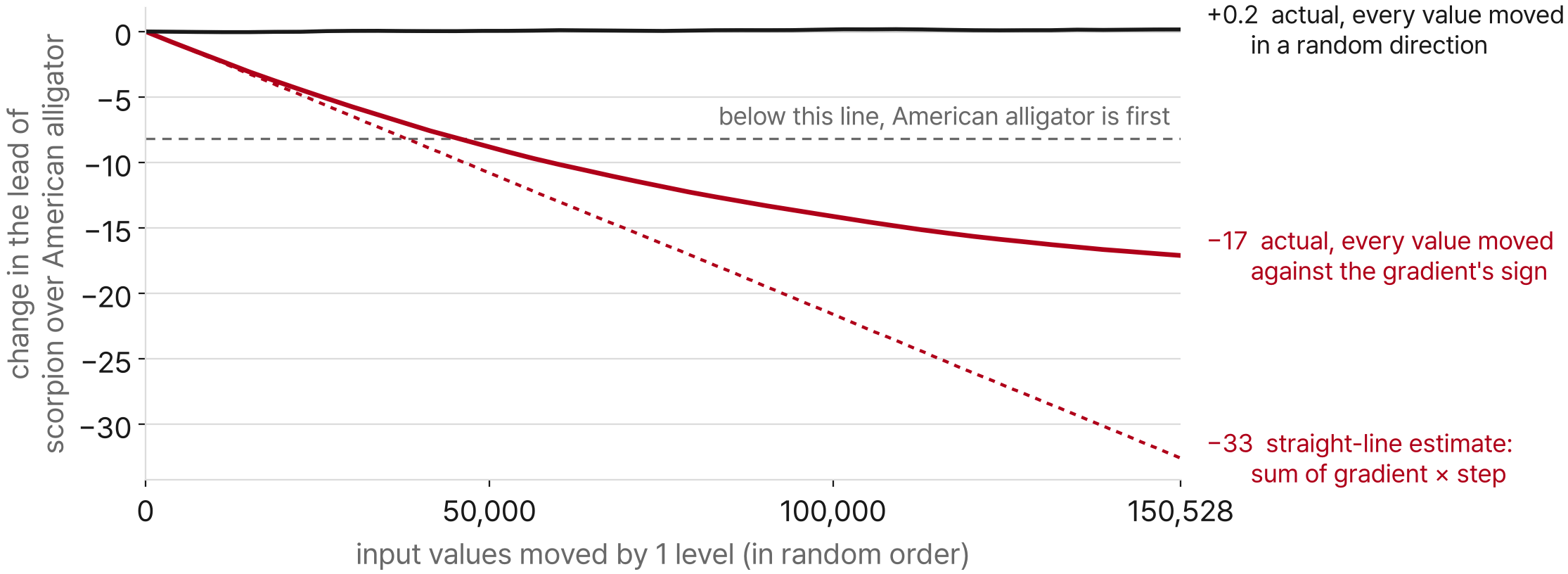
toaster, changed photo



score 31.2

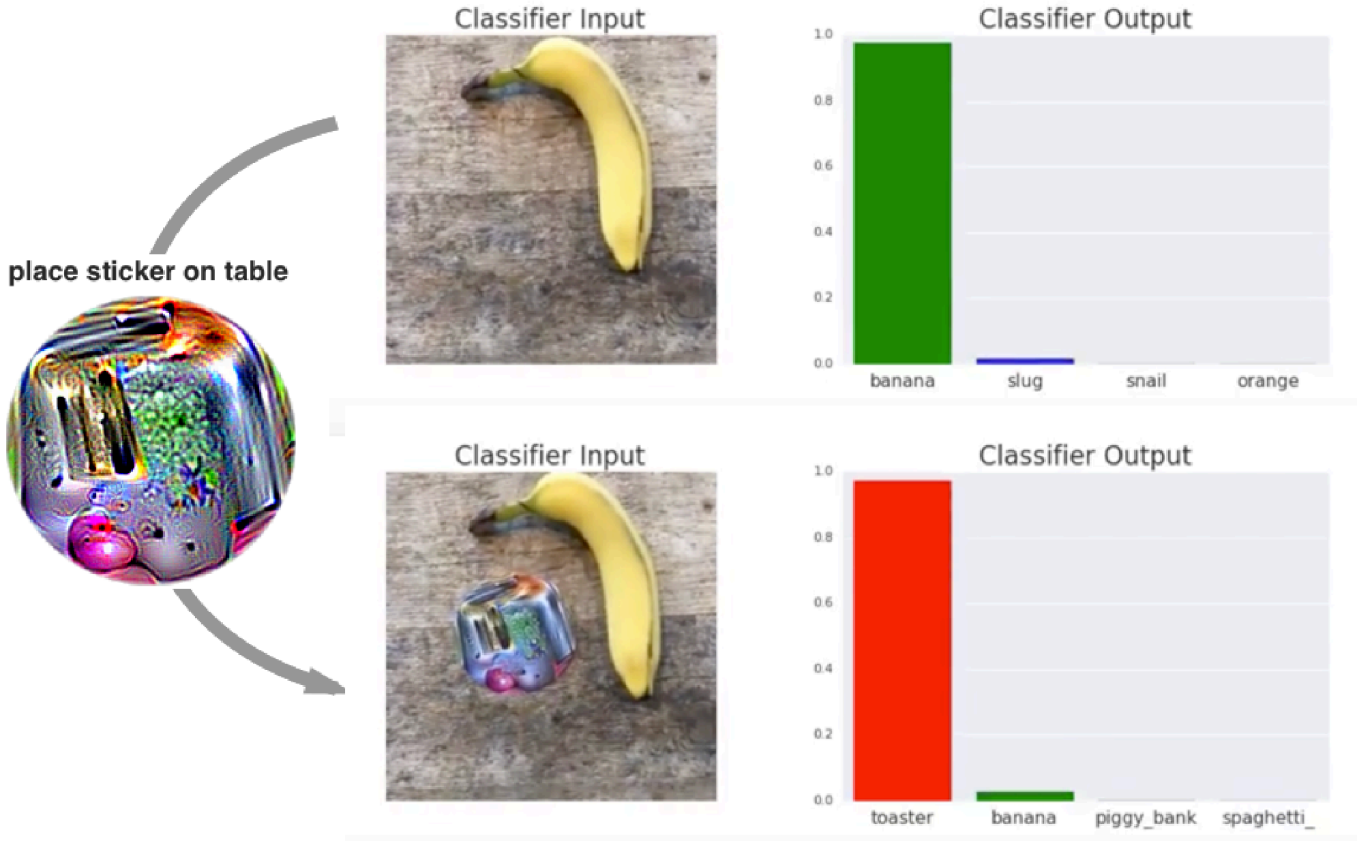
Many Small Changes Add Up

Scorpion photo from the demo: 3 × 224 × 224 = 150,528 input values



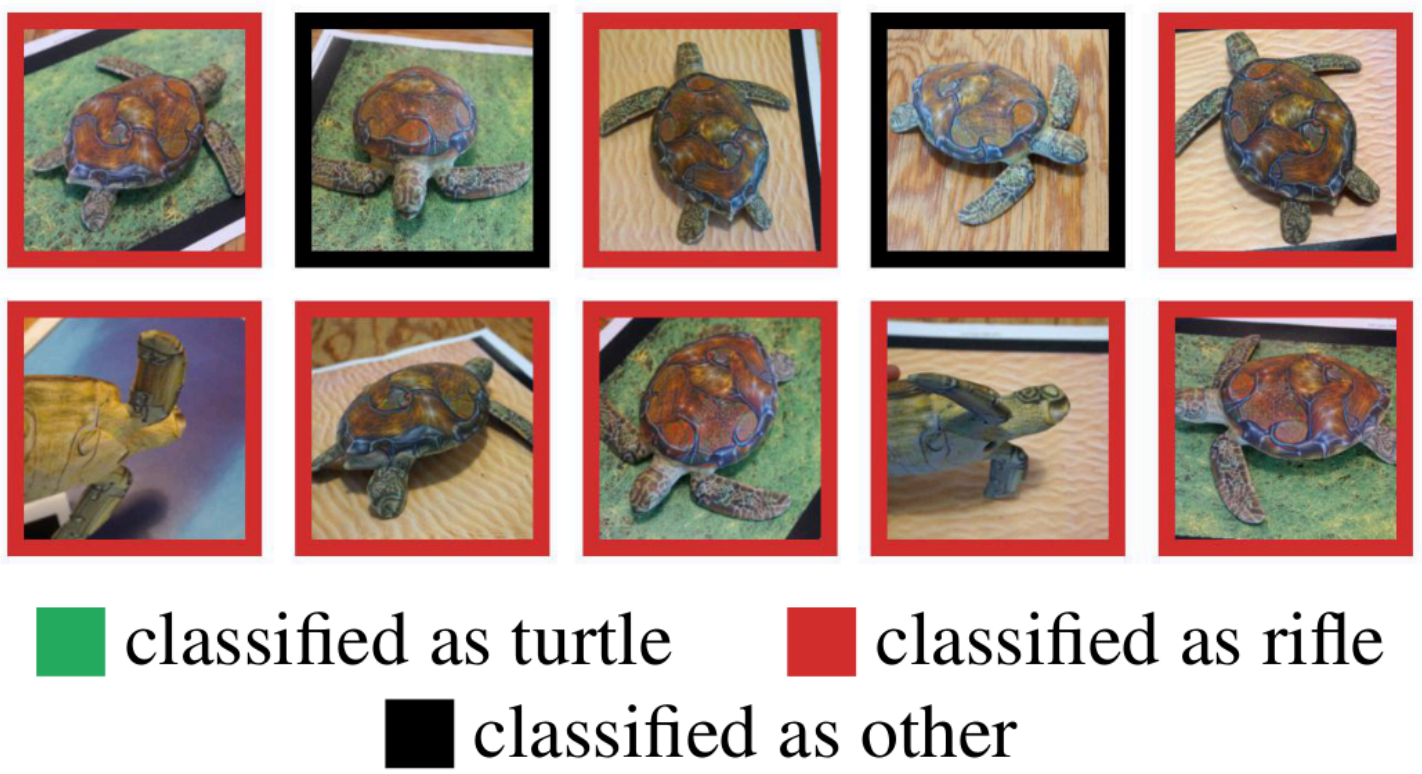
Adversarial Patch

Brown, Mané, Roy, Abadi, Gilmer (2017), Adversarial Patch, arXiv



An Adversarial Turtle

Athalye, Engstrom, Ilyas, Kwok (2018), Synthesizing Robust Adversarial Examples, ICML

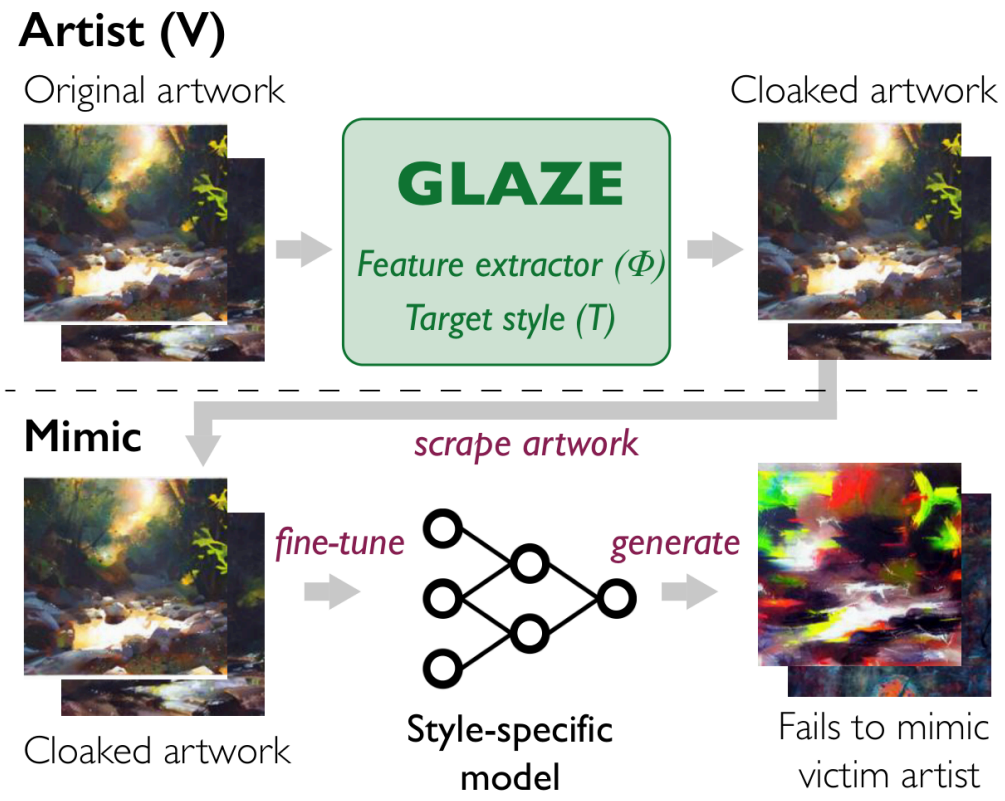


An Adversarial Stop Sign

Eykholt et al. (2018), Robust Physical-World Attacks on Deep Learning Visual Classification, CVPR

Glaze

Shan, Cryan, Wenger, Zheng, Hanocka, Zhao (2023), Glaze: Protecting Artists from Style Mimicry by Text-to-Image Models, USENIX Security



Coming Up

- Assignment 1 due Friday, October 16, 22:00